ASB Module Map

Five bot-owned modules for the smart-glasses anomaly-detection project

This document splits the Anomaly Signature Bench (ASB, repo `asb-rig`) into **five bot-owned modules** so work can run in parallel without collisions. It defines each module's scope, inputs, outputs, dependencies and reason to exist, the shared contracts between them, who edits what, and a short starter brief per bot. **Nothing has been built, bought, sent or installed.** It maps the v1 build spec (Phase 0–1) and the 15 modules being added in the v2 spec; where v2 wording differs once it is final, the Spec/Docs bot reconciles this map (Section 11).

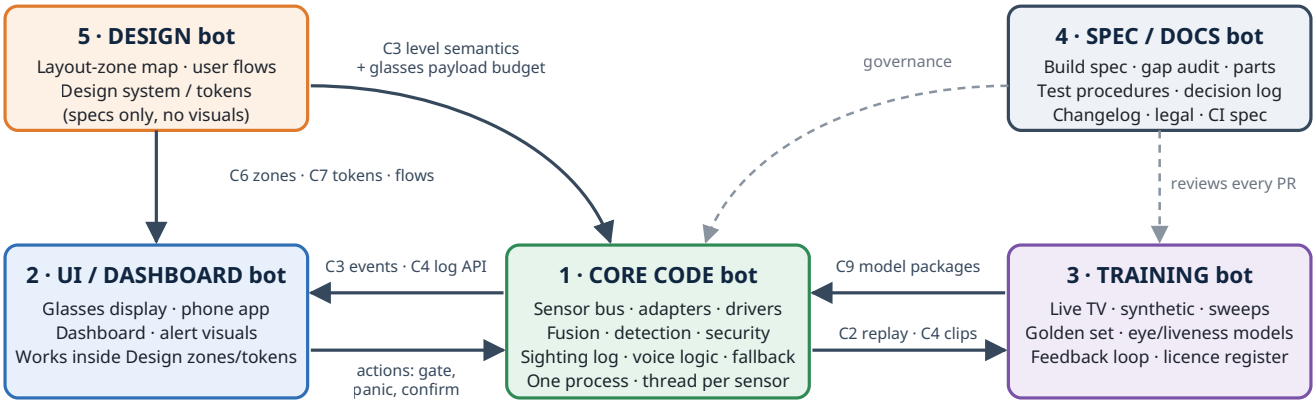
At a glance

Bot (tag)	Owns	Does not own
1 · Core code `[core]`	Sensor bus, adapter layer, drivers, features, detection logic, fusion layer, security code, sighting-log code, voice-output logic, baselines, Android fallback runtime, headless runner hooks.	Screens, layout, colours; model training; documents.
2 · UI / dashboard `[ui]`	Implementation of the glasses display, phone app and dashboard *inside the Design bot's zones and tokens*; alert visuals; map and timeline views; gate and panic UX hooks.	Layout and design-system decisions (now Design); detection or storage logic (Core).
3 · Training pipeline `[train]`	Live-TV, synthetic, recorded-sweep and golden-dataset pipelines; eye-behavior and liveness models; baseline and drift statistics; feedback loop; data licence register.	Runtime code; UI; spec text.
4 · Spec / docs `[docs]`	Build spec, gap audit, parts list, test procedures, decision log, changelog, legal/ethics checklist, macOS CI and local build spec, contract versioning.	Code, models, visuals.
5 · Graphic design `[design]`	Layout-zone map, user flows, design system (tokens, alert semantics, accessibility, glasses constraints). **Specifies the map only; produces no mockups.**	Implementation (UI); final pixel art or icon drawing.

Standing guardrails (apply to every bot, every module)

1. **Open-source only.** Allowed: MIT, BSD, Apache-2.0, ISC, PSF, MPL-2.0. Denied: AGPL, non-commercial, "research only", proprietary models or datasets. Our code is Apache-2.0, in one repo, `asb-rig`, kept standalone.
2. **Passive RF only.** BLE advertisements and AP beacons in managed mode; no transmitting, no jamming, no monitor mode, no data frames.
3. **Phase 0–1 = private, consenting use only.** Own premises, adult volunteers with consent IDs. No strangers, no public spaces.
4. **Never label a real person "robot".** Output is "signal observed" plus benign causes; no class-label field in any live schema; vocabulary guard stays in code and tests.
5. **Nothing purchased, built or sent** without the owner's explicit approval. No ads, no accuracy claims, no cloud.
6. **Architecture constraint:** one process, one thread per sensor, every module emits a score in [0, 1] plus a health flag, one fusion layer combines them.

1. Overview: five modules and the contracts between them



Shared contracts and their single owner (details in Section 8)

C1 Score schema 0–1 + health flag — Core · C2 Sensor adapter interface — Core · C3 Event/alert schema — Core
C4 Sighting-log schema — Core · C5 Golden-dataset format — Training · C6 Layout-zone map — Design
C7 Design tokens — Design · C8 Versioning and change policy — Spec/Docs · C9 Model-package manifest — Training

Solid arrows = data/artifact flow. Dashed grey = governance (review, versioning, requirements). Nothing here is built, bought or sent.

How the pieces connect. Design decides *where things go and how they look in rules*; UI builds inside those zones; Core produces the events and logs that fill the zones; Training supplies models and datasets that Core loads and tests against; Spec/Docs writes down the rules, tests everything against pass/fail criteria, and versions the contracts. The detection runtime stays **one process with a thread per sensor and a fusion layer** (on the Pi rig and, as a camera-only node, on Android). The phone app and dashboard are separate programs that only talk to it through contracts C3 and C4 and the encrypted channel.

Interface (arrow)	Carries	Direction
Core → UI	Alert events (C3), sighting-log queries and history (C4), render-ready thermal frames, sensor-health	Core is producer; UI is consumer
UI → Core	User actions: gate result, panic gesture, kill switch, confirm/reject, voice settings	UI produces; Core executes
Design → UI	Zone map (C6), design tokens (C7), user flows, copy rules	Design produces; UI consumes only
Design → Core	Alert-level semantics review, glasses payload and refresh budget	Design requests; Core confirms feasibility
Training → Core	Model packages (C9), fusion weights, baseline and drift parameters	Training produces; Core loads
Core → Training	Replay/headless runner, recorded-sweep export, clip export hook (C2, C4)	Core produces; Training consumes
Spec/Docs ↔ all	Requirements, guardrails, test criteria, changelog, contract versions (C8)	Spec reviews every PR

2. Module 1 · Core code bot `[core]`

Field	Definition
Scope: in	• **Sensor bus and adapter layer**: one adapter per device (thermal, NoIR camera, IR LED driver, BLE, Wi-Fi beacons, mmWave, IMU, SDR, microphone, GPS), a bounded queue per sensor thread, clock sync, health flags, a replay adapter that reads recorded data. • **Drivers and features**: glint, thermal ROI, RF summary, vital-motion band; eye-behavior and liveness *feature capture* at the rates Training needs. • **Detection logic and fusion**: A1–A6 signature scoring; **liveness stacking logic** (liveness + voiceprint + RF + thermal agreement, each a 0–1 score); the single fusion layer that applies weights supplied by Training; sensor-unhealthy suppression. • **Signal fingerprint module**: per-emitter RF baseline, drift flagged as tampering, device-type identification. • **Security module code**: AES-256 at rest, Android Keystore, offline-only operation, constant-time crypto, encrypted rotating-key glasses-to-phone channel, hash-chained tamper-evident log, kill switch with remote wipe, panic-gesture handler. • **Sighting log code**: encrypted storage, auto-logged GPS, query API (offline OSM tiles are data the UI renders). • **Voice output logic**: threat tier (low / medium / confirmed), barge-in rules, speech request queue, text-to-speech adapter. • **Per-location baselines** and the new-object-at-known-location alert. • **Android fallback runtime**: any Android device with a camera as a detection node (camera-only adapter, same score contract). • **Headless test mode runner hooks**; guard code (vocabulary, consent, retention); model-package loader.
Scope: out	Screen layout, colours, widgets (Design/UI); model training and datasets (Training); spec text, CI *specification* (Docs); choosing alert wording tokens (Design, within the vocabulary guard).
Inputs	Raw sensor data and device drivers; model packages and fusion weights (C9 from Training); requirements and guardrails (Spec); level semantics and display payload budget (Design); user actions from UI (gate result, panic, confirm/reject, settings); offline map tile packs as plain data files (no network).
Outputs	**C1** score records; **C2** adapter interface and reference adapters; **C3** alert events; **C4** sighting-log schema and encrypted log; sweep-history query API; render-ready thermal frames (palette-agnostic); speech requests; recorded-sweep and clip export; headless runner (replays a dataset, emits scores and events, exits with pass/fail); Android fallback node; build scripts.
Depends on	← **Training** (models, weights, baselines); ← **Spec** (requirements, test criteria); ← **Design** (level semantics, payload budget: consult, not blocking); ← **UI** (action messages). Everyone else depends on Core for C1–C4.
Why its own bot	It is the only module that touches hardware, crypto and the real-time thread model, so one owner prevents timing, safety and security regressions from leaking in.

3. Module 2 · UI / dashboard bot `[ui]`

Field	Definition
Scope: in	Implementing, **strictly inside the Design bot's zone map (C6) and tokens (C7)**: glasses display renderer (alert strip, alert line, thermal view, health dots); phone app (live view, threat panel, voice controls, tabs, status strip); dashboard (sighting table, map, timeline, sensor health, review queue); **thermal grayscale default with colour overlay toggle**; **sweep history timeline** view; **offline OSM pin map view** with attribution line; **app-gate UX** (fingerprint or face unlock screens, lock states); **panic-gesture UX** and kill-switch confirmation screens as specified by Design; voice and alert visuals; confirm/reject controls for the feedback loop; Android screens for the fallback node.
Scope: out	**Layout and design-system decisions** (zones, colours, type, icons, spacing, alert semantics, flows: Design). Where a needed zone or token is missing, UI files a request to Design; it does not invent one. Also out: detection, storage, crypto, voice logic (Core); models (Training).
Inputs	C6 zone map, C7 tokens, user-flow specs, copy rules (Design); C3 alert events and C4 log/history API, health flags, thermal frames (Core); acceptance criteria and accessibility checks (Spec); offline tile packs.
Outputs	Glasses UI package; phone app (APK built from the same commit as the macOS artifacts); dashboard; mock-event harness so UI can be built before Core is finished; zone-conformance report (each screen element mapped to a zone ID); UI test results; user actions sent to Core.
Depends on	← **Design** (hard: every screen), ← **Core** (C3, C4; mock events until ready), ← **Spec** (acceptance criteria). Supplies actions to Core and Training.
Why its own bot	Rendering, platform quirks and device testing are a different skill set and change fast, but must not change detection behaviour or redefine the visual rules.

4. Module 3 · Training pipeline bot `[train]`

Field	Definition
Scope: in	Four **multi-source learning modules**: (a) **live TV** capture and clip extraction; (b) **synthetic renders** (open-licensed tools only); (c) **recorded sweeps** from the rig with consenting volunteers and props; (d) the **golden dataset** (frozen, versioned, subject-split). Models: **eye-behavior** (micro-saccades, pupil response, corneal reflection), **liveness**, new-object-at-location, fusion weights (logistic regression, leave-one-subject-out), per-emitter drift thresholds. **Feedback loop**: confirmed sightings become labeled golden-dataset clips. The data licence and rights register.
Scope: out	Runtime code, drivers, UI, spec prose. Training never ships code into the live process except as a model package Core loads.
Guardrails (hard)	• **Live TV and any recorded footage need a rights/licence review before use and are private-research-only**: stored in a *restricted tier*, never redistributed, never in a released dataset, and not used for any model that is shipped or published unless the register records clearance. • **Only commercially licensable models and data**; no non-commercial licences, no "research only" weights, no AGPL training tools bundled into outputs. • **Biometric data (eye, voiceprint) stays on-device and consented**; no upload, deletion on withdrawal, never used to identify a person. • **Labels describe the physical test object** (person, mannequin, powered mechanism, costume/prop, display screen). A real person is **never** labeled "robot"; the vocabulary guard's rules apply to datasets too. • **Eye-behavior and liveness are hypotheses**: every model ships with a model card stating the hypothesis, the **proxy tests** that exercise it (heated mannequin eye, glass-eye prop, photo or screen replay, silicone mask prop, human controls), and the false-positive cap (≤5% on human controls). No accuracy claim is made beyond measured results.
Inputs	Recorded sweeps and clip exports, replay runner (Core); consent records and guardrails (Spec); confirm/reject actions (UI) via Core's export hook; review-flow spec (Design); rights decisions (the owner).
Outputs	**C5** golden-dataset format and dataset releases (private); **C9** model packages with model cards and licence manifests; fusion weights; baseline and drift parameters; evaluation reports; data licence register; clip-labeling queue content for the dashboard review zone.
Depends on	**← Core** (replay runner, exports, model-adapter interface C2/C9), **← Spec** (guardrails, consent, test criteria), **← UI/Design** (feedback review flow). **→ Core** supplies models and weights.
Why its own bot	Data rights, labeling discipline and model evaluation need one gatekeeper who can say "this data may not be used", separate from the people who ship runtime code.

5. Module 4 · Spec / docs bot `[docs]`

Field	Definition
Scope: in	**Build spec** (currently v1; v2 in progress), **gap audit** (before/after), **parts list** (BOM), **wiring and power plan**, **test procedures with pass/fail criteria** including headless mode and the golden dataset, **decision log** (open and settled defaults), **changelog** (v1 to v2 onward), **legal/ethics checklist** (including OpenStreetMap attribution and the human-subject policy), **macOS CI and local build spec** (Android APK and macOS artifacts from the same commit), versioning of contracts (C8), keeping this module map current.
Scope: out	Writing application code, training models, producing visuals. The Spec bot writes the CI *specification*; Core implements the scripts.
Inputs	Everything the other bots produce (contract changes, test results, model cards, zone map and flows); the owner's decisions; source checks.
Outputs	Spec PDF + Markdown source; gap-audit table; BOM; decision log; changelog; test-procedure catalog with pass/fail thresholds; legal checklist; CI/local-build spec; `contracts/CHANGELOG`; PR review verdicts against the guardrail checklist.
Depends on	**← all bots** for facts (read-only feedback); **→ all bots** for requirements and acceptance criteria. Has no build-order blocker; it starts first and finishes last.
Why its own bot	A neutral owner of rules and acceptance tests keeps pass/fail thresholds from drifting toward whatever the builders happened to achieve.

6. Module 5 · Graphic design bot `[design]` (layout and design-system owner)

This bot specs the map. It does not draw visuals or mockups. Its job is to give every other bot a precise answer to "where does my piece plug in, and which rules apply?".

Field	Definition
Scope: in	**(a) Layout-zone map** covering the glasses display, phone app and dashboard: named zones, and where each key function lives (detection alerts, thermal overlay [grayscale default, colour toggle], threat-level indicators [low / medium / confirmed], sighting-log pins [offline OSM], voice controls, sweep history timeline, app gate, panic-gesture UX, health/sensor status). **(b) User flow** from first launch (consent, app-gate setup, glasses-to-phone pairing, calibration) through a sweep and the review/log step. **(c) Design system**: colour roles, type scale, icon style, spacing, alert-level semantics, accessibility and contrast rules, glasses-specific constraints (low brightness, glanceability). Copy rules for alert text (vocabulary-guard compatible).
Scope: out	Mockups, final artwork, icon drawing, implementation, animation code; detection and alert *logic* (Core); platform widgets (UI).
Inputs	Requirements, guardrails and accessibility acceptance criteria (Spec); C3 event schema and health flags, display payload limits (Core); feasibility feedback (UI); feedback-loop needs (Training).
Outputs	**C6** layout-zone map (zone IDs, surfaces, size classes, which fields render where, refresh budget); **C7** design-tokens file (token names, roles, values, glasses variants); user-flow specs (first launch, sweep, review, panic, kill switch); alert-level semantics sheet; accessibility checklist; copy deck rules; a zone-conformance review of UI pull requests.
Depends on	**← Core** (C3 fields, payload budget), **← Spec** (requirements, accessibility criteria), **← UI** (feasibility). **→ UI** (hard, everything), **→ Core** (level semantics, payload budget requests), **→ Training/Docs** (review flow, accessibility criteria).
Why its own bot	One owner of layout and tokens means the glasses, phone and dashboard stay consistent and the UI bot fills in blanks instead of inventing.

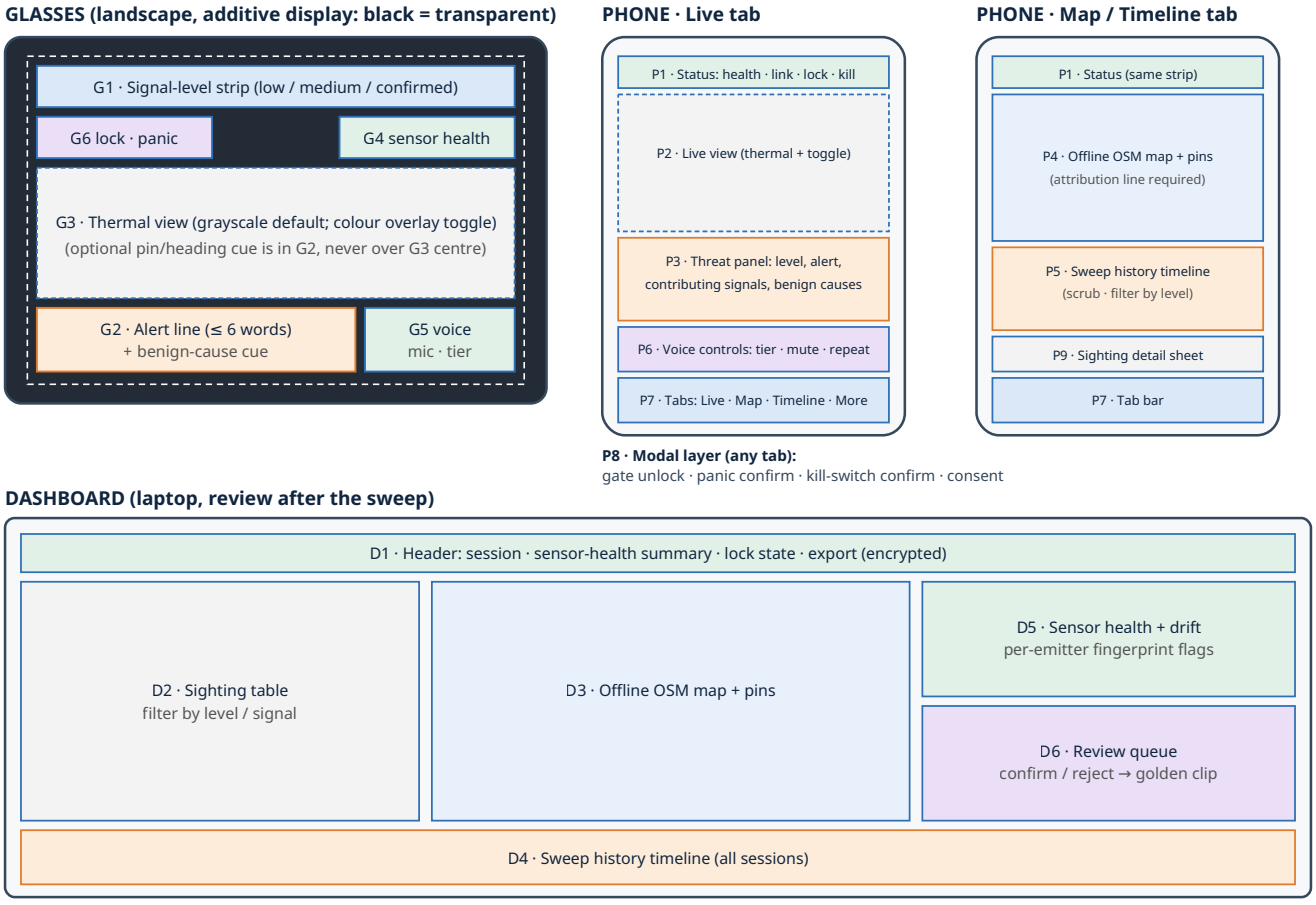
6.1 Design-system rules the Design bot must define (defaults; Design may refine within these limits)

[DEFAULT]

Area	Rule the token set must satisfy
Alert-level semantics	Three levels only: **low, medium, confirmed**. "Confirmed" means several independent sensors agree, **never** an identity. Each level has a distinct *shape, text word, tone and haptic pattern*; colour is a redundant cue only. Level names are used with "signal" (e.g. "Medium signal"), never "robot".
Colour	Alert-level colours are **reserved** and never used in the thermal colour overlay or decoration. Palette must stay legible for common colour-vision deficiencies. Dark-first theme.
Contrast / accessibility	Text ≥ 4.5:1, large text and UI parts ≥ 3:1 (WCAG 2.2 AA); touch targets ≥ 48 dp; every alert reachable by one hand; scalable text; no information by colour alone.
Typography	One open-licensed sans family (licence check by Docs), a short scale (≤ 6 sizes), tabular figures for numbers; glasses text is one size up from phone text.
Icon style	One open-licensed set, single stroke weight, glyph + word for any safety function (lock, panic, kill, voice).
Spacing / motion	One spacing scale; motion minimal and never flashing (no more than 3 flashes per second); reduced-motion respected.
Glasses constraints	Additive display, so black is transparent: use sparse light-on-dark, **low default brightness**, never fill the field of view; **glanceable**: ≤ 6 words per alert line, readable in ≈ 1–2 s; keep the **centre of view clear** except the thermal window the wearer chooses; no scrolling, no small text, no pop-ups during movement; thermal view is **grayscale by default** with a colour overlay toggle. Numeric limits are [DEFAULT] and to be validated on the actual display in Phase 2.

7. Layout-zone map (placement map, not a visual design)

> The figure below is a **placement map**: boxes and labels only, to show where each function lives. It is **not a visual design**: no colours, type or styling here are final (the shading only separates zone families). Real values come from the Design bot's C6 and C7.



7.1 Where each key function lives

Function	Glasses	Phone	Dashboard
Detection alerts	G2 alert line	P3 threat panel	D2 table
Threat-level indicator (low / medium / confirmed)	G1 strip	P3 (and P1 badge)	D2 filter, D4 marks
Thermal view, grayscale default + colour toggle	G3	P2	(frame in D2 detail)
Sighting-log pins, offline OSM	G2 "new sighting" cue only	P4, P9	D3
Voice controls and tier	G5	P6	settings only
Sweep history timeline	none	P5	D4
App gate (fingerprint / face unlock)	G6 lock state	P8 modal at launch and on resume	D1 lock state
Panic-gesture UX	G6 armed indicator, gesture on glasses	P8 confirm modal, P1 kill-state	D1 state
Health / sensor status	G4 dots	P1 strip	D5, D1
Feedback (confirm / reject)	none	P9 sheet	D6 review queue

7.2 User flow the Design bot must specify (first launch to review)

Step	Flow	Notes for Design / UI / Core
1	**Welcome and guardrails notice**: private, consenting use; "signal observed, not an identification"	Copy from Spec; one screen
2	**Consent**: operator confirms own policy; consent ID per human subject (v1 rule R-09)	Core enforces; UI shows state
3	**App-gate setup**: fingerprint or face unlock; device-credential fallback	Biometric class and fallback are Core/Spec defaults (Section 10)
4	**Pair glasses to phone**: short verification code shown on both, then rotating-key channel starts	Core owns key exchange; Design owns the screens
5	**Calibration and sensor check**: health of each sensor, thermal calibration prompts, per-location baseline capture	Health flags from C1
6	**Sweep**: live view, signal-level strip, alert line, voice tier, panic gesture available at all times	Glanceability rules apply
7	**Review**: sighting list and map, detail sheet, confirm/reject	Feeds Training via Core export
8	**Log and timeline**: history timeline, encrypted export, session end	Kill switch / wipe flow reachable from More tab

8. Shared contracts (single owner each)

Contracts live in `contracts/` in `asb-rig`. Each has a **named owner bot** who alone edits it; consumers may only propose changes by PR. Fields below are the *required* content, not code.

ID	Contract	Required content	Owner	Consumers
C1	**Score schema**	Per-sensor/module record: sensor or module ID, **score as a float in [0, 1]**, **health flag** (ok / degraded / failed), monotonic and UTC timestamps, schema version, optional reason codes. Nothing outside 0–1; no class labels.	**Core**	UI, Training, Docs, Design
C2	**Sensor adapter interface**	Lifecycle (open, start, stop, close), capability declaration, record format, timestamp rules, health reporting, a replay-source flag so recorded data is indistinguishable from live to the fusion layer. Same contract for hardware, Android camera and replay.	**Core**	Training, Docs
C3	**Event / alert schema**	Event ID, time, signature code (A1–A6 and new modules), **level (low / medium / confirmed)**, score, contributing signals, benign-cause note, location ID, health snapshot. **No class-label field.** Level semantics co-reviewed by Design.	**Core**	UI, Design, Training, Docs
C4	**Sighting-log schema**	Encrypted record: event reference, auto GPS fields, location ID, confirm status, consent flag, **hash-chain fields** (previous hash, record hash), tamper status, retention date. Includes history query semantics for the timeline.	**Core**	UI, Training, Docs
C5	**Golden-dataset format**	Clip manifest: clip ID, **source tier** (own-recorded, synthetic, restricted-licensed), rights record, consent reference, sensor streams with SHA-256, lab-label table (object type only), subject-based split, dataset version.	**Training**	Core (headless), Docs
C6	**Layout-zone map**	Zone IDs per surface (G, P, D families), size classes, which C3/C1 fields render in each zone, refresh budget, modal layer rules, glasses payload limits.	**Design**	UI, Core, Docs
C7	**Design tokens**	Token names and roles: colour, type, spacing, icon set, motion, alert-level semantic tokens, glasses brightness variants.	**Design**	UI, Docs
C8	**Versioning and change policy**	Semantic version per contract; consumers must support the current and previous version; breaking change = major bump, all consumer bots sign off, plus the owner if it touches a guardrail; every change logged in `contracts/CHANGELOG`.	**Docs**	all
C9	**Model-package manifest**	Model file + model card (hypothesis, proxy tests, metrics), licence manifest, training-data tier statement, input spec, calibrated **0–1 output**, version; fusion-weights file format.	**Training**	Core, Docs

Direction rule. Contracts flow downstream only (owner to consumer). A consumer that needs a change files a request with the owner; it does not fork the contract.

9. Dependency and handoff matrix

Read each row as "**from** this bot, **to** the column bot" (what is handed over). "—" means no direct handoff.

From ↓ / To →	Core	UI	Training	Docs	Design
Core	—	C3 events, C4 log + history API, thermal frames, health	Replay runner, sweep and clip export, model adapter spec	Interface notes, test hooks, build scripts	C3 fields, payload budget, feasibility
UI	Actions: gate, panic, confirm/reject, settings	—	Confirm/reject decisions (via Core)	UI test results, APK build config	Feasibility feedback, zone-gap requests
Training	C9 models, fusion weights, baselines, drift thresholds	Review-queue content (clip previews)	—	Model cards, licence register, eval reports	Review-flow constraints
Docs	Requirements, test criteria, CI spec, guardrail verdicts	Acceptance + accessibility criteria	Consent and legal rules, test criteria	—	Requirements, copy rules, accessibility criteria
Design	Level semantics review, payload budget request	C6, C7, flows, copy rules, zone-conformance review	Review-flow spec	Flows for test procedures	—

Blocking vs non-blocking. Hard blockers: UI needs Design's C6/C7 (for anything beyond mock screens) and Core's C3/C4 (mock events allowed meanwhile). Core needs nothing blocking except guardrails. Training needs Core's replay runner and export format. All other links are "consult".

10. Build order and parallel-work plan (planning only; nothing starts without the owner's go)

Wave	Core	UI	Training	Docs	Design
0 · Freeze rules	Review C1–C4 drafts	—	Draft C5, licence register	Freeze guardrails, C8, ownership rules; reconcile v2 spec	Collect requirements; draft zone families
1 · Contracts and skeletons	C1, C2 final; sensor bus, replay adapter, one reference adapter	Mock-event harness; wait on C6	C5 final; data intake and rights review; synthetic pipeline	Test-plan skeleton; CI/local build spec	**C6 v1, C7 v1, user flows** (UI's gate)
2 · Core and views	Fusion, guard, security, sighting log, C3/C4 final	Glasses and phone shell in zones; thermal view; status strip	Recorded-sweep pipeline; baseline and drift stats	Gap audit, parts list, wiring plan, security test procedures	Alert semantics sheet; accessibility checklist; review flow
3 · Signal modules	Fingerprint, liveness stacking, voice tiers, baselines, fallback node	Map, timeline, gate UX, panic UX, review queue	Eye-behavior and liveness models (hypotheses + proxy tests); fusion weights	Test procedures for each module	Zone-conformance reviews; fallback layout variant
4 · Integrate and prove	Headless runner on golden dataset; hardening	Conformance fixes; dashboard	Golden dataset v1; feedback loop	Pass/fail run, changelog, go/no-go G0	Final tokens; contrast audit

What runs in parallel from day one: Spec/Docs, Design and Training intake (none of them needs Core code). **Critical path:** Design C6/C7 → UI screens, and Core C1–C4 → everything else. **Security gate:** nothing records real data until Core's security module and the consent guard pass Docs' tests. Waves carry no dates; the owner sets pace and approvals. Purchases and hardware builds stay behind decision D-01.

Defaults settled here (override any by telling the owner's Grok Bot) [DEFAULT]

ID	Default
M-01	Design bot's C6/C7 are the only source of layout and tokens; UI never invents zones.
M-02	The "one process, thread per sensor" rule applies to the detection runtime (Pi rig and Android fallback node). Phone app and dashboard are separate programs reading C3/C4.
M-03	Live-TV and other recorded footage stay in a restricted tier, private-research-only, excluded from shipped models unless the register shows clearance.

ID	Default
M-04	App gate uses Android's strongest available biometric class with device credential as fallback; face unlock counts only if the device reports it as strong (**[UNVERIFIED on the target phone]**).
M-05	Offline OSM map displays "© OpenStreetMap contributors" attribution; Docs adds it to the legal checklist.
M-06	Until the owner approves the repo as public, CI runs locally on the owner's Mac; hosted CI only on free public-repo runners (Section 11).

11. Ownership and conflict rules

Path in `asb-rig` (one repo, Apache-2.0, standalone)	Owner bot	Others may
`asb/sensors, sync, features, fusion, guard, security, store, voice, baselines, fallback, headless`	**Core**	Propose by PR or issue
`ui/glasses, ui/phone, ui/dashboard`	**UI**	Design reviews zone conformance
`design/` (zone map, flows, tokens source, copy rules)	**Design**	Propose by issue
`training/`, dataset manifests, model cards (data and weights are **not** in git)	**Training**	Core reviews interfaces
`docs/`, gap audit, BOM, test procedures, decision log, changelog, legal, `contracts/CHANGELOG`	**Docs**	Others append facts via PR
`contracts/` files	**Owner named in Section 8**	Consumers propose
`.github/workflows`, build scripts	**Core** implements, **Docs** specifies	UI supplies Android build config

Rules.

1. **PR-only.** No direct pushes to the main branch, no force-pushes on shared branches. One PR touches one bot's paths; cross-boundary work is two PRs or an issue to the owner. PR titles start with the bot tag: `[core]`, `[ui]`, `[train]`, `[docs]`, `[design]`.
2. **Review gates (all required):** (a) reviewed by a bot other than the author; (b) license audit passes (no AGPL, no non-commercial, GPL only as a separate executable); (c) vocabulary-guard test and consent test pass; (d) contract-version check passes; (e) UI PRs pass Design's zone-conformance review; (f) Docs verifies the guardrail checklist. **the owner's approval is required** for: anything purchased or built as hardware, human-subject policy, public or stranger use, rights clearance of restricted data, changes to the vocabulary blocklist, any guardrail change.
3. **No paid GitHub Actions.** Checked today against GitHub's docs: standard hosted runners (including macOS) are free and unlimited **on public repositories**; private-repo macOS minutes and all larger runners are billed. Therefore: no larger runners, no private-repo hosted macOS; set the Actions spending limit to $0; until the repo is approved as public, run the macOS and APK jobs on the owner's own Mac with the same scripts.
4. **Contract disputes** go to Docs to log, then to the owner. Nobody edits a contract they do not own.
5. **Data and secrets never go in git.** Datasets, clips, keys, logs, GPS traces stay on encrypted local storage. The repo contains manifests only.
6. **Nothing from or into other projects.** No shared code, data or models; other projects release rules do not apply here and vice versa.
7. **Temp-file hygiene.** Each bot cleans its scratch files after a run and keeps the box above 8 GiB free.
8. **A change that adds a sensor or module** must show: an adapter (C2), a 0–1 score with health flag (C1), a fusion entry, a test procedure, and a zone or "no UI" statement, in that order.

12. RACI for the 15 new modules

A = accountable and does the main work (exactly one per row); **R** = does a defined part; **C** = consulted before changes; **I** = informed. Docs additionally owns the spec section, test procedure and changelog entry for every row; shown as R only where its part is substantial.

#	Module	Core	UI	Train	Docs	Design	Primary owner + notes on support
1	Eye-behavior analysis (micro-saccades, pupil, corneal reflection)	R	I	**A**	C	I	**Train**: models + proxy tests (hypothesis). Core: high-rate eye-region capture + score adapter
2	Liveness stacking (liveness + voiceprint + RF + thermal)	**A**	I	R	C	C	**Core**: stacking logic. Train: liveness model + weights. Design: how contributing signals are shown
3	Signal fingerprint (RF baseline, drift = tamper, device type)	**A**	I	R	C	C	**Core**. Train: drift thresholds from recorded sweeps. Design: tamper alert wording
4	Security (AES-256, Keystore, offline, constant-time, rotating-key channel, hash-chained log, kill switch, panic)	**A**	R	C	R	R	**Core**: code. UI: lock, kill, panic screens. Design: panic-gesture UX. Docs: threat model + tests
5	Sighting log (encrypted, auto GPS, offline OSM pins)	**A**	R	C	C	C	**Core**: log + API. UI: map and list views. Design: pin zones
6	Voice output, threat-tiered barge-in	**A**	R	I	C	R	**Core**: tier and barge-in logic. Design: tier semantics and phrase length. UI: controls
7	Per-location baselines + new-object alert	**A**	C	R	C	C	**Core**: baseline store and alert. Train: baseline and new-object statistics. UI shows the alert in Design's zones
8	Thermal grayscale default + colour overlay toggle	R	**A**	I	C	R	**UI**: renderer. Core: palette-agnostic frames. Design: toggle spec, reserved alert colours
9	Headless test mode + golden dataset	R	I	R	**A**	I	**Docs**: test plan and criteria. Core: runner hooks. Train: golden dataset (C5)
10	macOS CI (APK + macOS artifacts, same commit)	R	R	I	**A**	I	**Docs**: CI/local build spec. Core: scripts. UI: Android build config. No paid Actions
11	Sensor bus / driver adapter layer	**A**	I	C	C	I	**Core**. Train: replay adapter requirements
12	Feedback loop (confirmed sightings to golden clips)	R	R	**A**	C	R	**Train**: clip labeling + dataset. Core: export hook. UI/Design: confirm/reject flow
13	Android fallback node (any camera device)	**A**	R	C	C	C	**Core**: runtime. UI: screens. Train: lighter models. Design: camera-only layout variant
14	Sweep history timeline	R	**A**	I	C	R	**UI**: view. Core: history query API (C4). Design: zone and interaction spec
15	App gate (fingerprint / face unlock)	R	**A**	I	C	R	**UI**: gate screens. Core: Keystore binding. Design: setup flow and lock states

13. Starter briefs (copy-paste prompts, plain text, no code)

Each brief is meant to be pasted to a new bot. All five share the guardrails; the closing lines are identical on purpose.

> **1 · Core code bot**
> You are the Core code bot for the asb-rig project (Anomaly Signature Bench smart-glasses detection, Apache-2.0, a separate repo standalone). You own the sensor bus, adapter layer, drivers, features, detection logic, the single fusion layer, security module code, sighting-log code, voice-output logic, per-location baselines, the Android fallback runtime and the headless test runner hooks. Keep one process, one thread per sensor, every module emitting a 0-to-1 score with a health flag. You own contracts C1 to C4; read the Module Map v1.1 before starting. Do not do screen layout (Design), screens (UI), model training (Training) or spec text (Docs). Work by pull request only; request changes from other bots by issue.

> **2 · UI / dashboard bot**
> You are the UI bot for asb-rig. You implement the glasses display, the phone app and the dashboard strictly inside the zone map (C6) and design tokens (C7) published by the Design bot. You do not decide layout, colours, type, icons or alert semantics; if a zone or token is missing, file a request to the Design bot. You build thermal grayscale with a colour overlay toggle, the sweep history timeline, the offline map with pins and OpenStreetMap attribution, the app gate screens, the panic and kill-switch confirmation screens, and the review controls. Use mock events until Core's C3 and C4 are ready. Pull requests only, tagged [ui].

> **3 · Training pipeline bot**
> You are the Training pipeline bot for asb-rig. You build four data sources (live TV, synthetic renders, recorded sweeps, the golden dataset), the eye-behavior and liveness models, baseline and drift statistics, and the feedback loop that turns confirmed sightings into labeled golden clips. You own contracts C5 and C9. Hard rules: live TV and recorded footage need a rights and licence review first and are private-research-only in a restricted tier; use only commercially licensable models and data, no non-commercial licences; biometric data stays on-device and consented; labels describe the physical test object and a real person is never labeled a robot. Treat eye-behavior and liveness as hypotheses with proxy tests and model cards. Pull requests only, tagged [train]; data and weights never go in git.

> **4 · Spec / docs bot**
> You are the Spec and docs bot for asb-rig. You own the build spec, gap audit (before and after), parts list, wiring and power plan, test procedures with pass or fail criteria, the decision log, the changelog, the legal and ethics checklist, the macOS CI and local build spec, and contract versioning (C8). You review every pull request against the guardrail checklist and keep the Module Map current. Settle open decisions as recorded defaults; never buy, build or send anything. Mark unverified facts as unverified and cite sources with access dates. Pull requests only, tagged [docs].

> **5 · Graphic design bot (layout and design-system owner)**
> You are the Graphic design bot for asb-rig. You specify, but do not draw, the visual layout of the whole system so every other bot knows where its pieces plug in. Produce: (a) the layout-zone map (C6) for the glasses display, phone app and dashboard, with zones for detection alerts, thermal view with grayscale default and colour toggle, low/medium/confirmed indicators, sighting pins, voice controls, sweep timeline, app gate, panic gesture and sensor health; (b) the user flow from first launch (consent, gate setup, pairing, calibration) through a sweep to review; (c) the design system (C7): colour roles, type, icon style, spacing, alert semantics, accessibility, and glasses limits such as low brightness and glanceability. No mockups, no real-person "robot" wording. Review UI pull requests for zone conformance. Pull requests only, tagged [design].

Closing lines for every brief: open-source only; passive RF only; Phase 0 to 1 is private consenting use only; never label a real person "robot"; nothing is purchased, built or sent without the owner's approval; follow the one-process, thread-per-sensor, 0-to-1 score, fusion-layer architecture; ask the Spec/Docs bot to log any decision you need made.

14. Uncertain or open items

- **v2 spec still in progress.** The 15-module list is taken from the owner's request, not from the finished v2 text; Docs reconciles names and IDs when v2 is final.
- **Eye-behavior feasibility.** Micro-saccade measurement needs a high-rate eye camera; whether the Tier A NoIR camera or any glasses camera can do it is **unverified**. It stays a hypothesis with proxy tests.
- **Bot identity.** Whether the five bots will use separate GitHub identities or one shared identity is not decided; PR title tags are the fallback.

- **Design defaults** (word counts, flash limits, contrast, 48 dp targets) are my choices from common accessibility guidance; glasses values need validation on the real display.
- **Biometric strength** on the target phone ([UNVERIFIED], M-04) and the UI framework choice (license-checked by Docs) are left to the owning bots.
- **Repo visibility** (private or public) is not decided; it determines whether hosted CI is free (M-06).
- **Live-TV rights** are unreviewed; until cleared, the TV pipeline is a plan only.